M.Tech - Database Management Systems

Exam-Oriented Notes (5-7 Marks Questions)

#	Topic
1	SQL Statements (DDL, DML, DCL, TCL, DQL)
2	Triggers
3	Cursors
4	Normalization
5	Stored Procedures and Functions
B	Important Questions with Model Solutions (18 questions)

How to use: Each topic has Definition, Types/Syntax, Example, Advantages, Differences and a list of likely 5-7 mark questions. For 5 marks write definition + 3-4 points + 1 example. For 7 marks add syntax, diagram/table and differences.

1. SQL STATEMENTS

1.1 Definition

SQL (Structured Query Language) is the standard, declarative language used to create, manipulate, query and control data stored in a Relational DBMS. The user specifies *what* data is needed, not *how* to get it.

1.2 Categories of SQL Statements

Category	Full form	Purpose	Commands
DDL	Data Definition Language	Defines/changes structure (schema) of database objects	CREATE, ALTER, DROP, TRUNCATE, RENAME
DML	Data Manipulation Language	Inserts, modifies, deletes data in tables	INSERT, UPDATE, DELETE
DQL	Data Query Language	Retrieves data	SELECT
DCL	Data Control Language	Controls access and privileges	GRANT, REVOKE
TCL	Transaction Control Language	Manages transactions	COMMIT, ROLLBACK, SAVEPOINT

1.3 DDL Commands

```
CREATE TABLE Student (
    roll_no  INT PRIMARY KEY,
    name     VARCHAR(30) NOT NULL,
    dept_id  INT,
    marks    DECIMAL(5,2) CHECK (marks >= 0),
    FOREIGN KEY (dept_id) REFERENCES Dept(dept_id)
);
ALTER TABLE Student ADD email VARCHAR(40);      -- add column
ALTER TABLE Student MODIFY name VARCHAR(50);    -- change datatype
ALTER TABLE Student DROP COLUMN email;          -- remove column
TRUNCATE TABLE Student;   -- removes all rows, keeps structure
DROP TABLE Student;       -- removes structure + data
```

DELETE vs TRUNCATE vs DROP

Point	DELETE	TRUNCATE	DROP
Type	DML	DDL	DDL
Removes	Selected rows (WHERE)	All rows	Whole table (structure + data)
Rollback	Possible	Generally not possible	Not possible
Speed	Slow (logs each row)	Fast	Fast
Triggers	Fire	Do not fire	Do not fire

1.4 DML Commands

```
INSERT INTO Student VALUES (1,'Amit',10,78.5);
INSERT INTO Student(roll_no,name) VALUES (2,'Riya');
UPDATE Student SET marks = marks + 5 WHERE dept_id = 10;
DELETE FROM Student WHERE marks < 35;
```

1.5 DQL - SELECT Statement

General form (clauses execute in the order: FROM, WHERE, GROUP BY, HAVING, SELECT, ORDER BY):

```
SELECT   [DISTINCT] column_list / aggregate functions
FROM     table(s)
[WHERE   row condition]
[GROUP BY columns]
[HAVING  group condition]
[ORDER BY columns ASC|DESC];
```

Aggregate functions

COUNT(), SUM(), AVG(), MIN(), MAX(). Example:

```
SELECT dept_id, COUNT(*) AS total, AVG(marks) AS avg_marks
FROM   Student
GROUP  BY dept_id
HAVING AVG(marks) > 60
ORDER  BY avg_marks DESC;
```

WHERE vs HAVING: WHERE filters rows *before* grouping and cannot use aggregates; HAVING filters groups *after* grouping and can use aggregates.

Joins

Join	Result
INNER JOIN	Only matching rows from both tables
LEFT OUTER JOIN	All rows of left table + matching rows of right (NULL if none)
RIGHT OUTER JOIN	All rows of right table + matching rows of left
FULL OUTER JOIN	All rows of both tables, NULL where no match
CROSS JOIN	Cartesian product (m x n rows)
SELF JOIN	Table joined with itself using aliases

```
SELECT s.name, d.dept_name
FROM   Student s INNER JOIN Dept d ON s.dept_id = d.dept_id;
```

Subquery (Nested query)

```
-- students scoring above average
SELECT name FROM Student
WHERE marks > (SELECT AVG(marks) FROM Student);

-- correlated subquery: highest marks in each dept
SELECT name FROM Student s
WHERE marks = (SELECT MAX(marks) FROM Student WHERE dept_id = s.dept_id);
```

1.6 DCL and TCL

```
GRANT  SELECT, INSERT ON Student TO user1;
REVOKE INSERT ON Student FROM user1;

INSERT INTO Student VALUES (5,'Raj',10,60);
SAVEPOINT sp1;
UPDATE Student SET marks = 0;
ROLLBACK TO sp1;     -- undo update only
COMMIT;              -- make changes permanent
```

1.7 Constraints

- **NOT NULL** - value cannot be empty
- **UNIQUE** - no duplicate values

- **PRIMARY KEY** - unique + not null, identifies a row
- **FOREIGN KEY** - enforces referential integrity
- **CHECK** - restricts values by condition
- **DEFAULT** - assigns a default value

1.8 Views and Indexes (often asked with SQL)

```
CREATE VIEW top_students AS
  SELECT name, marks FROM Student WHERE marks > 75;
CREATE INDEX idx_name ON Student(name);
```

A **view** is a virtual table based on a query (security, simplicity). An **index** is a data structure (B-tree) that speeds up searching.

> **Likely questions:** (1) Explain categories of SQL statements with examples. (2) Differentiate DELETE, TRUNCATE and DROP. (3) Explain types of joins with examples. (4) Explain GROUP BY and HAVING. (5) Write note on nested/correlated subqueries. (6) Explain integrity constraints in SQL.

2. TRIGGERS

2.1 Definition

A **trigger** is a special type of stored program (PL/SQL block) that is **automatically executed (fired) by the DBMS** in response to an event such as INSERT, UPDATE or DELETE on a table. It cannot be called explicitly. Triggers follow the **ECA (Event-Condition-Action)** model.

2.2 ECA Model

Component	Meaning	Example
Event	Operation that activates trigger	INSERT / UPDATE / DELETE on Employee
Condition	Optional check (WHEN clause)	salary > 50000
Action	Block of code executed	Insert a row in audit table

2.3 Syntax

```
CREATE [OR REPLACE] TRIGGER trigger_name
{BEFORE | AFTER | INSTEAD OF}
{INSERT | UPDATE [OF column] | DELETE}
ON table_name
[FOR EACH ROW]
[WHEN (condition)]
BEGIN
    -- trigger body (PL/SQL statements)
END;
```

2.4 Types of Triggers

(a) Based on timing

- **BEFORE trigger** - fires before the DML statement executes (validation, setting default values).
- **AFTER trigger** - fires after the DML completes (auditing, logging).
- **INSTEAD OF trigger** - used on views; replaces the DML action so that complex views can be updated.

(b) Based on level

- **Row-level trigger** - uses FOR EACH ROW; fires once for every row affected. Can use :OLD and :NEW.
- **Statement-level trigger** - fires once per SQL statement regardless of number of rows.

(c) Based on event

INSERT trigger, UPDATE trigger, DELETE trigger, and combined triggers (INSERT OR UPDATE OR DELETE). Other kinds: DDL triggers (CREATE/DROP) and system/database triggers (LOGON, STARTUP).

2.5 :OLD and :NEW qualifiers

Event	:OLD	:NEW
INSERT	NULL	New values being inserted
UPDATE	Values before update	Values after update
DELETE	Values of row being deleted	NULL

2.6 Examples

Example 1 - Audit trigger (AFTER UPDATE):

```
CREATE OR REPLACE TRIGGER trg_salary_audit
AFTER UPDATE OF salary ON Employee
FOR EACH ROW
BEGIN
    INSERT INTO Emp_Audit(emp_id, old_sal, new_sal, changed_on)
    VALUES (:OLD.emp_id, :OLD.salary, :NEW.salary, SYSDATE);
END;
```

Example 2 - Validation trigger (BEFORE INSERT):

```
CREATE OR REPLACE TRIGGER trg_check_marks
BEFORE INSERT OR UPDATE ON Student
FOR EACH ROW
BEGIN
    IF :NEW.marks < 0 OR :NEW.marks > 100 THEN
        RAISE_APPLICATION_ERROR(-20001, 'Marks must be between 0 and 100');
    END IF;
END;
```

2.7 Managing Triggers

```
ALTER TRIGGER trg_check_marks DISABLE;
ALTER TRIGGER trg_check_marks ENABLE;
DROP  TRIGGER trg_check_marks;
```

2.8 Advantages and Disadvantages

Advantages	Disadvantages
Enforces complex business rules and integrity automatically	Hard to debug; execution is invisible to the user
Maintains audit trail / logs of changes	Overuse slows down DML operations
Keeps derived data and replicas in sync	Cascading triggers can cause confusion / infinite loops
Centralised logic in the database, not in each application	Mutating-table error in row-level triggers

2.9 Trigger vs Procedure

Trigger	Procedure
Executed automatically on event	Executed explicitly using CALL / EXEC
Cannot take parameters	Can take IN, OUT, IN OUT parameters
Cannot contain COMMIT/ROLLBACK (normally)	Can contain transaction control
Tied to a table/view	Independent schema object

Likely questions: (1) What is a trigger? Explain types with syntax and example. (2) Explain ECA model. (3) Differentiate row-level and statement-level triggers. (4) Write a trigger to maintain audit of salary changes. (5) Differentiate trigger and stored procedure.

3. CURSORS

3.1 Definition

A **cursor** is a pointer to a private memory area (context area) created by the DBMS to hold the result set of a SQL query. It allows a program to **process the rows returned by a query one at a time**. This solves the mismatch between set-oriented SQL and row-oriented procedural languages (PL/SQL).

3.2 Types of Cursors

A. Implicit Cursor

Automatically created by Oracle for every DML statement (INSERT, UPDATE, DELETE) and SELECT INTO. The programmer cannot control it, but can read its attributes using the name **SQL**.

B. Explicit Cursor

Declared and controlled by the programmer for queries returning **multiple rows**.

3.3 Cursor Attributes

Attribute	Description
%FOUND	TRUE if the last fetch / DML affected at least one row
%NOTFOUND	TRUE if the last fetch returned no row
%ISOPEN	TRUE if cursor is open (always FALSE for implicit cursor after execution)
%ROWCOUNT	Number of rows fetched / affected so far

3.4 Steps (Life cycle) of an Explicit Cursor

Step	Statement	Action
1. DECLARE	CURSOR c IS SELECT ...;	Defines cursor with its query
2. OPEN	OPEN c;	Executes query, populates the result set, pointer before first row
3. FETCH	FETCH c INTO var;	Reads current row into variables and moves pointer ahead
4. CLOSE	CLOSE c;	Releases memory / context area

3.5 Example - Explicit Cursor

```
DECLARE
    CURSOR c_emp IS
        SELECT emp_id, name, salary FROM Employee WHERE dept_id = 10;
    v_id    Employee.emp_id%TYPE;
    v_name  Employee.name%TYPE;
    v_sal   Employee.salary%TYPE;
BEGIN
    OPEN c_emp;
    LOOP
        FETCH c_emp INTO v_id, v_name, v_sal;
        EXIT WHEN c_emp%NOTFOUND;
        DBMS_OUTPUT.PUT_LINE(v_id || ' ' || v_name || ' ' || v_sal);
    END LOOP;
    CLOSE c_emp;
END;
```

Cursor FOR loop (simplest - opens, fetches and closes automatically):

```
BEGIN
    FOR rec IN (SELECT name, salary FROM Employee) LOOP
        DBMS_OUTPUT.PUT_LINE(rec.name || ' earns ' || rec.salary);
    END LOOP;
END;
```

3.6 Implicit Cursor Example

```
BEGIN
    UPDATE Employee SET salary = salary * 1.10 WHERE dept_id = 10;
    IF SQL%FOUND THEN
        DBMS_OUTPUT.PUT_LINE(SQL%ROWCOUNT || ' rows updated');
    ELSE
        DBMS_OUTPUT.PUT_LINE('No rows updated');
    END IF;
END;
```

3.7 Parameterized Cursor and REF Cursor

```
CURSOR c_dept(p_id NUMBER) IS
    SELECT name FROM Employee WHERE dept_id = p_id;
-- usage:  OPEN c_dept(20);  or  FOR r IN c_dept(20) LOOP ... END LOOP;
```

- **Parameterized cursor** - accepts arguments, so same cursor can be reused with different values.
- **REF CURSOR** - a dynamic cursor variable whose query is decided at run time; can be passed between programs.
- **FOR UPDATE cursor** - locks the fetched rows so they can be updated using WHERE CURRENT OF.

3.8 Implicit vs Explicit Cursor

Point	Implicit	Explicit
Created by	DBMS automatically	Programmer
Used for	DML, single-row SELECT INTO	Multi-row SELECT
Control	No OPEN/FETCH/CLOSE	Full control: OPEN, FETCH, CLOSE
Name	SQL	User-defined
Code size	Less	More

3.9 Advantages

- Row-by-row processing of multi-row results
- Better control over data retrieval and processing logic
- Allows navigation, locking and updating current row
- Bridges set-oriented SQL with procedural logic

Disadvantage: slower than set-based SQL for large data because of row-by-row processing and extra memory use.

Likely questions: (1) What is a cursor? Explain types with example. (2) Explain cursor attributes. (3) Explain steps to use an explicit cursor. (4) Differentiate implicit and explicit cursor. (5) Write PL/SQL program using cursor to display employee details.

4. NORMALIZATION

4.1 Definition

Normalization is the systematic process of organising the attributes and tables of a relational database to **minimise data redundancy** and eliminate **insertion, update and deletion anomalies**, by decomposing a relation into smaller well-structured relations using **functional dependencies**.

4.2 Need / Anomalies

Consider relation **STUDENT_COURSE(Roll, Name, Course, Instructor, Phone)** where student details repeat for each course.

- **Insertion anomaly** - cannot add a new course until some student enrols in it.
- **Deletion anomaly** - deleting the last student of a course also deletes the instructor information.
- **Update anomaly** - changing an instructor's phone requires updating many rows; missing one causes inconsistency.

4.3 Functional Dependency (FD)

X -> Y means: if two tuples have the same value of X, they must have the same value of Y (X *determines* Y).

- **Trivial FD:** Y is a subset of X (e.g., {Roll, Name} -> Name).
- **Non-trivial FD:** Y is not a subset of X.
- **Partial dependency:** non-key attribute depends on part of a composite key.
- **Transitive dependency:** X -> Y and Y -> Z hence X -> Z, where Y is non-key.

Armstrong's Axioms

- **Reflexivity:** if Y is a subset of X then X -> Y
- **Augmentation:** if X -> Y then XZ -> YZ
- **Transitivity:** if X -> Y and Y -> Z then X -> Z
- Derived: Union, Decomposition, Pseudotransitivity

Closure and Candidate Key

The closure X^+ is the set of all attributes functionally determined by X. If X^+ contains all attributes of R, X is a **super key**; a minimal super key is a **candidate key**.

4.4 Normal Forms

Normal Form	Condition	Removes
1NF	All attributes atomic (single-valued); no repeating groups	Multivalued / composite attributes
2NF	In 1NF + no partial dependency (every non-key attribute fully dependent on whole primary key)	Partial dependency
3NF	In 2NF + no transitive dependency. For every FD X -> A: X is a super key OR A is a prime attribute	Transitive dependency
BCNF	For every non-trivial FD X -> Y, X must be a super key	Anomalies due to FDs where determinant is not a key
4NF	In BCNF + no non-trivial multivalued dependency (MVD)	Multivalued dependency

Normal Form	Condition	Removes
5NF	In 4NF + no join dependency (lossless decomposition only through candidate keys)	Join dependency

4.5 Worked Example

Unnormalized table

Roll	Name	Courses	Instructor
1	Amit	DBMS, OS	Rao, Sen
2	Riya	DBMS	Rao

Step 1 - 1NF (make values atomic)

Roll	Name	Course	Instructor
1	Amit	DBMS	Rao
1	Amit	OS	Sen
2	Riya	DBMS	Rao

Primary key = (Roll, Course).

Step 2 - 2NF (remove partial dependency)

Name depends only on Roll (part of key). Decompose:

```
STUDENT(Roll, Name)               -- Roll -> Name
ENROLL(Roll, Course, Instructor)  -- (Roll, Course) -> Instructor
```

Step 3 - 3NF (remove transitive dependency)

In ENROLL: (Roll, Course) -> Course and Course -> Instructor, so Instructor depends transitively on the key. Decompose:

```
STUDENT(Roll, Name)
ENROLL(Roll, Course)
COURSE(Course, Instructor)        -- Course -> Instructor
```

BCNF Example

R(Student, Subject, Teacher) with FDs: (Student, Subject) -> Teacher and Teacher -> Subject. Teacher is not a super key, so R violates BCNF. Decompose into R1(Teacher, Subject) and R2(Student, Teacher).

4.6 Properties of Good Decomposition

- **Lossless-join:** joining decomposed relations gives back exactly the original relation (no spurious tuples). Test: for R1, R2, common attributes must be a key of R1 or R2.
- **Dependency preservation:** all original FDs can be enforced on the decomposed relations without joining. (3NF always guarantees both; BCNF guarantees lossless but may lose dependency preservation.)

4.7 3NF vs BCNF

3NF	BCNF
Allows X -> A if A is a prime attribute	Requires X to be a super key for every FD
Always lossless and dependency preserving	Always lossless, may not preserve dependencies

3NF	BCNF
Some redundancy may remain	Removes almost all redundancy from FDs
Weaker	Stronger (every BCNF is in 3NF)

4.8 Denormalization

Deliberately introducing redundancy (merging tables) to improve read/query performance, avoiding costly joins, at the cost of storage and update anomalies. Used in data warehouses/OLAP.

4.9 Advantages

- Reduces redundancy and saves storage
- Removes insertion, deletion, update anomalies
- Improves data integrity and consistency
- Easier maintenance and flexible design

Likely questions: (1) What is normalization? Explain 1NF, 2NF, 3NF with example. (2) Explain BCNF and differentiate from 3NF. (3) Explain functional dependency and Armstrong's axioms. (4) Explain anomalies with example. (5) Explain lossless-join and dependency-preserving decomposition. (6) Normalize given relation up to 3NF/BCNF.

5. STORED PROCEDURES AND FUNCTIONS

5.1 Definition

A **stored procedure** is a named, precompiled block of PL/SQL (or SQL/PSM) statements stored in the database that performs a specific task and is executed on demand. A **function** is similar but **must return a single value** and can be used inside SQL expressions. Together with triggers and packages they are called **stored (named) subprograms**.

5.2 PL/SQL Block Structure

```
DECLARE      -- optional: variables, cursors
BEGIN        -- mandatory: executable statements
EXCEPTION    -- optional: error handling
END;
```

5.3 Procedure

Syntax

```
CREATE [OR REPLACE] PROCEDURE proc_name
    (p1 [IN|OUT|IN OUT] datatype, ...)
IS | AS
    -- local declarations
BEGIN
    -- statements
EXCEPTION
    -- handlers
END proc_name;
```

Parameter modes

Mode	Meaning
IN (default)	Value passed into procedure; read-only
OUT	Procedure returns value to caller
IN OUT	Passed in, modified and returned back

Example

```
CREATE OR REPLACE PROCEDURE raise_salary
    (p_id IN NUMBER, p_percent IN NUMBER) IS
BEGIN
    UPDATE Employee
    SET    salary = salary + salary * p_percent / 100
    WHERE  emp_id = p_id;
    IF SQL%NOTFOUND THEN
        DBMS_OUTPUT.PUT_LINE('Employee not found');
    END IF;
    COMMIT;
END raise_salary;
/
-- calling:
EXEC raise_salary(101, 10);
-- or inside PL/SQL:  raise_salary(101, 10);
```

Procedure with OUT parameter:

```
CREATE OR REPLACE PROCEDURE get_salary
    (p_id IN NUMBER, p_sal OUT NUMBER) IS
BEGIN
    SELECT salary INTO p_sal FROM Employee WHERE emp_id = p_id;
EXCEPTION
```

```
    WHEN NO_DATA_FOUND THEN p_sal := 0;
END;
```

5.4 Function

Syntax

```
CREATE [OR REPLACE] FUNCTION func_name
    (p1 IN datatype, ...)
RETURN return_datatype
IS | AS
BEGIN
    -- statements
    RETURN value;
EXCEPTION
    -- handlers
END func_name;
```

Example

```
CREATE OR REPLACE FUNCTION get_bonus (p_sal IN NUMBER)
RETURN NUMBER IS
BEGIN
    RETURN p_sal * 0.15;
END get_bonus;
/
-- calling:
SELECT name, salary, get_bonus(salary) FROM Employee;
```

Factorial function example:

```
CREATE OR REPLACE FUNCTION fact (n NUMBER) RETURN NUMBER IS
    f NUMBER := 1;
BEGIN
    FOR i IN 1..n LOOP
        f := f * i;
    END LOOP;
    RETURN f;
END;
```

5.5 Procedure vs Function

Point	Procedure	Function
Return value	May return zero, one or many values (through OUT)	Must return exactly one value using RETURN
Purpose	Perform an action / business operation	Compute and return a value
Call in SQL	Cannot be called in SELECT	Can be called in SELECT, WHERE, etc.
Call method	EXEC / CALL or PL/SQL statement	As expression: x := func(a);
DML inside	Allowed freely	Restricted if called from SQL query
RETURN keyword	Optional (exit only)	Mandatory with data type

5.6 Advantages of Stored Procedures / Functions

- **Reusability** - written once, used by many applications
- **Performance** - precompiled; reduces network traffic (one call instead of many SQL statements)
- **Security** - users get EXECUTE privilege without direct table access
- **Maintainability** - business logic centralised in database
- **Modularity** and easier debugging

- **Data integrity** through consistent business rules

5.7 Disadvantages

- Business logic tied to a vendor-specific language (PL/SQL, T-SQL) - less portable
- Harder to version-control and debug than application code
- Heavy logic in DB increases server load

5.8 Exception Handling

```
EXCEPTION
    WHEN NO_DATA_FOUND   THEN ... -- SELECT INTO returned no row
    WHEN TOO_MANY_ROWS   THEN ... -- SELECT INTO returned many rows
    WHEN ZERO_DIVIDE     THEN ...
    WHEN OTHERS          THEN DBMS_OUTPUT.PUT_LINE(SQLERRM);
```

5.9 Managing Subprograms

```
DROP PROCEDURE raise_salary;
DROP FUNCTION  get_bonus;
SELECT * FROM USER_SOURCE WHERE name = 'RAISE_SALARY';
```

Likely questions: (1) Explain stored procedure with syntax and example. (2) Differentiate procedure and function. (3) Explain parameter modes IN, OUT, IN OUT. (4) Write a function to calculate factorial / bonus. (5) Explain advantages of stored subprograms. (6) Explain exception handling in PL/SQL.

PART B: IMPORTANT QUESTIONS WITH MODEL SOLUTIONS

Paper pattern (20 marks): A paper like this usually has 3 questions of about 5-7 marks (e.g. 7 + 7 + 6) or four 5-mark questions, often with internal choice. Each solution below is written in **exam-ready format** with a suggested mark split. Write the same points, headings, one diagram/table and one example to score full marks.

Section A: SQL Statements

Q1. Explain the different categories of SQL statements with suitable examples. [7 Marks]

Answer: SQL commands are classified into five categories (definition = 1 mark, categories + examples = 5 marks, summary = 1 mark).

Category	Purpose	Example
DDL	Create / change structure of objects	CREATE TABLE Emp(id INT PRIMARY KEY, name VARCHAR(20)); ALTER TABLE Emp ADD salary INT; DROP TABLE Emp;
DML	Insert / modify / delete data	INSERT INTO Emp VALUES(1,'Amit',40000); UPDATE Emp SET salary=45000 WHERE id=1; DELETE FROM Emp WHERE id=1;
DQL	Retrieve data	SELECT name, salary FROM Emp WHERE salary > 30000;
DCL	Control access rights	GRANT SELECT ON Emp TO user1; REVOKE SELECT ON Emp FROM user1;
TCL	Control transactions	COMMIT; ROLLBACK; SAVEPOINT s1;

Conclusion: DDL commands auto-commit, whereas DML changes are permanent only after COMMIT.

Q2. Differentiate between DELETE, TRUNCATE and DROP. [5 Marks]

Point	DELETE	TRUNCATE	DROP
Category	DML	DDL	DDL
Effect	Removes selected/all rows	Removes all rows	Removes table structure and data
WHERE clause	Allowed	Not allowed	Not applicable
Rollback	Yes	Generally no	No
Triggers	Fire	Do not fire	Do not fire
Speed	Slowest	Fast	Fast
Table structure	Retained	Retained	Deleted

Marks: any 5 correct differences = 5 marks.

Q3. Consider Employee(emp_id, name, salary, dept_id) and Dept(dept_id, dept_name). Write SQL queries for: (a) names of employees with salary above 50000, (b) department-wise average salary, (c) departments having more than 5 employees, (d) second highest salary, (e)

employee name with department name, (f) departments having no employees. [6 Marks]

Answer: (1 mark per query)

```
(a) SELECT name FROM Employee WHERE salary > 50000;

(b) SELECT dept_id, AVG(salary) AS avg_sal
    FROM Employee GROUP BY dept_id;

(c) SELECT dept_id, COUNT(*) AS cnt
    FROM Employee GROUP BY dept_id HAVING COUNT(*) > 5;

(d) SELECT MAX(salary) FROM Employee
    WHERE salary < (SELECT MAX(salary) FROM Employee);

(e) SELECT e.name, d.dept_name
    FROM Employee e INNER JOIN Dept d ON e.dept_id = d.dept_id;

(f) SELECT d.dept_name FROM Dept d
    WHERE d.dept_id NOT IN (SELECT dept_id FROM Employee
                            WHERE dept_id IS NOT NULL);
    -- or: Dept d LEFT JOIN Employee e ... WHERE e.emp_id IS NULL
```

Q4. Explain the different types of joins in SQL with examples and output. [7 Marks]

Answer: A join combines rows of two or more tables based on a related column (definition = 1 mark, each join with output = 1 mark each, 6 joins).

Tables used:

Emp: eid	name	did
1	Amit	10
2	Riya	20
3	Raj	NULL

Dept: did	dname
10	IT
20	HR
30	Sales

Join (query pattern)	Output
INNER JOIN FROM Emp e JOIN Dept d ON e.did=d.did	Amit-IT, Riya-HR (only matches)
LEFT OUTER JOIN	Amit-IT, Riya-HR, Raj-NULL (all of Emp)
RIGHT OUTER JOIN	Amit-IT, Riya-HR, NULL-Sales (all of Dept)
FULL OUTER JOIN	Amit-IT, Riya-HR, Raj-NULL, NULL-Sales
CROSS JOIN	3 x 3 = 9 rows (Cartesian product)
SELF JOIN	Table joined with itself, e.g. employee with manager: SELECT e.name, m.name FROM Emp e JOIN Emp m ON e.mgr_id = m.eid;

Q5. Differentiate WHERE and HAVING clauses with example. [5 Marks]

WHERE	HAVING
Filters individual rows	Filters groups
Applied before GROUP BY	Applied after GROUP BY
Cannot use aggregate functions	Can use aggregate functions
Can be used without GROUP BY	Normally used with GROUP BY

```
SELECT dept_id, AVG(salary)
FROM   Employee
WHERE  salary > 20000          -- row filter
GROUP  BY dept_id
HAVING AVG(salary) > 40000;    -- group filter
```

Section B: Triggers

Q6. What is a trigger? Explain its types, syntax and advantages with an example. [7 Marks]

Answer: Definition (1): A trigger is a stored PL/SQL block that executes automatically when a specified event (INSERT, UPDATE, DELETE) occurs on a table or view. It works on the Event-Condition-Action (ECA) model.

Types (2):

- **Timing:** BEFORE, AFTER, INSTEAD OF (for views)
- **Level:** Row-level (FOR EACH ROW, uses :OLD/:NEW) and Statement-level (once per statement)
- **Event:** INSERT, UPDATE, DELETE triggers; also DDL and system triggers

Syntax (1):

```
CREATE [OR REPLACE] TRIGGER name
{BEFORE | AFTER | INSTEAD OF} {INSERT | UPDATE | DELETE}
ON table_name [FOR EACH ROW] [WHEN (condition)]
BEGIN
   -- action
END;
```

Example (2): log every deleted employee

```
CREATE OR REPLACE TRIGGER trg_emp_delete
AFTER DELETE ON Employee
FOR EACH ROW
BEGIN
    INSERT INTO Emp_Log VALUES (:OLD.emp_id, :OLD.name, SYSDATE);
END;
```

Advantages (1): automatic enforcement of business rules, auditing, integrity, replication/synchronisation, centralised logic.

Q7. Write a trigger to prevent the balance of an account from becoming negative. [5 Marks]

```
CREATE OR REPLACE TRIGGER trg_check_balance
BEFORE UPDATE OF balance ON Account
FOR EACH ROW
BEGIN
    IF :NEW.balance < 0 THEN
        RAISE_APPLICATION_ERROR(-20010,
            'Insufficient balance: withdrawal not allowed');
    END IF;
END;
/
-- Test:
```

```
UPDATE Account SET balance = balance - 5000 WHERE acc_no = 101;
-- Error is raised if the new balance would be below zero and
-- the update is rolled back.
```

Explanation: It is a BEFORE row-level trigger, so it fires before each row is changed. :NEW.balance holds the value after update. RAISE_APPLICATION_ERROR stops the statement.

Q8. Differentiate between row-level and statement-level triggers. Also differentiate trigger and procedure. [5 Marks]

Row-level trigger	Statement-level trigger
FOR EACH ROW is specified	FOR EACH ROW is not specified
Fires once for every affected row	Fires once per SQL statement
Can access :OLD and :NEW	Cannot access :OLD / :NEW
Used for auditing, validation of each row	Used for security checks, logging statement

Trigger	Procedure
Fired automatically by an event	Called explicitly by user/program
No parameters	Accepts IN/OUT/IN OUT parameters
Attached to table or view	Independent schema object

Section C: Cursors

Q9. What is a cursor? Explain implicit and explicit cursors with attributes. Write a PL/SQL program using an explicit cursor. [7 Marks]

Answer: Definition (1): A cursor is a pointer to the private SQL work area that stores the result set of a query, so rows can be processed one by one.

Types (2): Implicit cursors are created automatically for DML and SELECT INTO (accessed as SQL%attribute). **Explicit** cursors are declared by the programmer for multi-row queries.

Attributes (1): %FOUND, %NOTFOUND, %ISOPEN, %ROWCOUNT.

Steps (1): DECLARE, OPEN, FETCH, CLOSE.

Program (2): Display name and salary of all employees earning above 30000.

```
DECLARE
    CURSOR c1 IS SELECT name, salary FROM Employee WHERE salary > 30000;
    v_name Employee.name%TYPE;
    v_sal  Employee.salary%TYPE;
BEGIN
    OPEN c1;
    LOOP
        FETCH c1 INTO v_name, v_sal;
        EXIT WHEN c1%NOTFOUND;
        DBMS_OUTPUT.PUT_LINE(v_name || ' - ' || v_sal);
    END LOOP;
    DBMS_OUTPUT.PUT_LINE('Rows fetched: ' || c1%ROWCOUNT);
    CLOSE c1;
END;
```

Q10. Differentiate between implicit and explicit cursors. Write a note on cursor FOR loop. [5 Marks]

Implicit	Explicit
Created automatically by DBMS	Defined by programmer
For DML and single-row SELECT INTO	For queries returning multiple rows
Named SQL	User-defined name
No OPEN, FETCH, CLOSE needed	OPEN, FETCH, CLOSE required

Cursor FOR loop: it automatically opens the cursor, fetches each row into a record variable and closes the cursor at the end, so less code and no chance of forgetting CLOSE.

```
FOR r IN (SELECT name FROM Employee) LOOP
    DBMS_OUTPUT.PUT_LINE(r.name);
END LOOP;
```

Section D: Normalization

Q11. What is normalization? Explain 1NF, 2NF and 3NF with example. [7 Marks]

Answer: Definition (1): Normalization is the process of decomposing relations using functional dependencies to reduce redundancy and avoid insertion, update and deletion anomalies.

Given: ORDER(order_id, product_id, product_name, customer_id, customer_name, qty). FDs: (order_id, product_id) -> qty; product_id -> product_name; order_id -> customer_id; customer_id -> customer_name. Key: (order_id, product_id).

1NF (1 mark): all attributes must be atomic. If a row stores 'Pen, Book' in product_name, split it into separate rows. Relation above is already in 1NF.

2NF (2 marks): 1NF + no partial dependency. product_name depends only on product_id, and customer_id depends only on order_id (parts of the key). Decompose:

```
PRODUCT(product_id, product_name)
ORDERS(order_id, customer_id, customer_name)
ORDER_ITEM(order_id, product_id, qty)
```

3NF (2 marks): 2NF + no transitive dependency. In ORDERS, order_id -> customer_id -> customer_name is transitive. Decompose:

```
CUSTOMER(customer_id, customer_name)
ORDERS(order_id, customer_id)
PRODUCT(product_id, product_name)
ORDER_ITEM(order_id, product_id, qty)
```

Result (1): every relation is now in 3NF; redundancy of product and customer names is removed.

Q12. Given R(A, B, C, D, E) with FDs F = {AB -> C, B -> D, C -> E}. Find the candidate key and decompose R into 3NF/BCNF. [7 Marks]

Answer: Step 1: Find candidate key (2 marks). Attributes never on the right side: A and B, so they must be in the key.

```
(AB)+ = {A,B}
      AB -> C   => {A,B,C}
      B  -> D   => {A,B,C,D}
      C  -> E   => {A,B,C,D,E}   = all attributes
A+ = {A}   B+ = {B,D}   (neither is a key alone)
So the candidate key = AB. Prime attributes: A, B.
```

Step 2: Check 2NF (2 marks). B -> D is a partial dependency (D depends on part of key AB). Decompose:

```
R1(B, D)          B -> D
```

```
R2(A, B, C, E)      AB -> C, C -> E
```

Step 3: Check 3NF/BCNF (2 marks). In R2, C -> E is a transitive dependency (C is not a super key, E is non-prime). Decompose R2:

```
R3(C, E)            C -> E
R4(A, B, C)         AB -> C
```

Step 4: Final result (1 mark). R1(B,D), R3(C,E), R4(A,B,C). In each relation the determinant of every FD is a key, so all relations are in **BCNF**. Decomposition is lossless (common attributes B, C are keys of R1, R3 respectively) and preserves all dependencies.

Q13. Explain insertion, deletion and update anomalies with example. How does normalization remove them? [5 Marks]

Relation EMP_DEPT(emp_id, emp_name, dept_id, dept_name, dept_head):

emp_id	emp_name	dept_id	dept_name	dept_head
1	Amit	D1	IT	Rao
2	Riya	D1	IT	Rao
3	Raj	D2	HR	Sen

- **Insertion anomaly:** A new department D3 cannot be stored until at least one employee is assigned to it (emp_id would be NULL, but it is a key).
- **Deletion anomaly:** Deleting employee Raj (the only employee of D2) also deletes the information of HR department and its head.
- **Update anomaly:** If the head of IT changes, rows of Amit and Riya must both be updated; missing one gives inconsistent data.

Solution: Normalize into EMP(emp_id, emp_name, dept_id) and DEPT(dept_id, dept_name, dept_head). Department details are now stored once, so all three anomalies disappear.

Q14. Differentiate between 3NF and BCNF. Explain lossless decomposition. [5 Marks]

3NF	BCNF
For FD X -> A: X is super key OR A is prime attribute	For every FD X -> Y: X must be a super key
Always lossless and dependency preserving	Always lossless; dependency preservation not guaranteed
May retain some redundancy	Removes redundancy caused by FDs
Less strict	More strict (every BCNF relation is in 3NF)

Lossless decomposition: R is decomposed into R1 and R2 such that natural join of R1 and R2 gives back exactly R (no extra or missing tuples). Condition: (R1 intersection R2) -> R1 or (R1 intersection R2) -> R2 must hold, i.e. the common attributes form a key of at least one of the relations.

Section E: Procedures and Functions

Q15. What is a stored procedure? Explain syntax, parameter modes and advantages with an example. [7 Marks]

Answer: Definition (1): A stored procedure is a named, precompiled PL/SQL block stored in the database that performs a specific task and is executed using EXEC/CALL.

Syntax (2):

```
CREATE [OR REPLACE] PROCEDURE name (param [IN|OUT|IN OUT] datatype)
IS
   -- declarations
BEGIN
   -- statements
EXCEPTION
   -- handlers
END;
```

Parameter modes (1): IN (input, default, read-only), OUT (returns value to caller), IN OUT (both input and output).

Example (2): procedure that finds salary and grade of an employee

```
CREATE OR REPLACE PROCEDURE emp_grade
    (p_id IN NUMBER, p_sal OUT NUMBER, p_grade OUT VARCHAR2) IS
BEGIN
    SELECT salary INTO p_sal FROM Employee WHERE emp_id = p_id;
    IF    p_sal >= 60000 THEN p_grade := 'A';
    ELSIF p_sal >= 40000 THEN p_grade := 'B';
    ELSE  p_grade := 'C';
    END IF;
EXCEPTION
    WHEN NO_DATA_FOUND THEN p_grade := 'NA';
END;
/
DECLARE s NUMBER; g VARCHAR2(2);
BEGIN
    emp_grade(101, s, g);
    DBMS_OUTPUT.PUT_LINE(s || ' ' || g);
END;
```

Advantages (1): reusability, better performance (precompiled, less network traffic), security (EXECUTE privilege only), modularity and easy maintenance.

Q16. Differentiate between procedure and function. Write a function to return the maximum of two numbers and use it in a PL/SQL block. [6 Marks]

Procedure	Function
May or may not return values (OUT params)	Must return exactly one value
Called as a statement (EXEC / CALL)	Called as an expression, also inside SELECT
RETURN only exits the procedure	RETURN with data type is mandatory
Used to perform actions	Used to compute a value

```
CREATE OR REPLACE FUNCTION max_of_two (a NUMBER, b NUMBER)
RETURN NUMBER IS
BEGIN
    IF a > b THEN RETURN a;
    ELSE RETURN b;
    END IF;
END;
/
DECLARE
    m NUMBER;
BEGIN
    m := max_of_two(25, 40);
    DBMS_OUTPUT.PUT_LINE('Maximum = ' || m);     -- Maximum = 40
END;
```

Q17. Write a PL/SQL procedure to transfer an amount from one account to another using a transaction. [5 Marks]

```
CREATE OR REPLACE PROCEDURE transfer
    (p_from IN NUMBER, p_to IN NUMBER, p_amt IN NUMBER) IS
    v_bal Account.balance%TYPE;
BEGIN
    SELECT balance INTO v_bal FROM Account WHERE acc_no = p_from;
    IF v_bal < p_amt THEN
        RAISE_APPLICATION_ERROR(-20020, 'Insufficient funds');
    END IF;
    UPDATE Account SET balance = balance - p_amt WHERE acc_no = p_from;
    UPDATE Account SET balance = balance + p_amt WHERE acc_no = p_to;
    COMMIT;                       -- both updates saved together
EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;                 -- undo if anything fails
        DBMS_OUTPUT.PUT_LINE('Transfer failed: ' || SQLERRM);
END;
```

Explanation: Both updates form one atomic transaction (debit + credit). If an error occurs, ROLLBACK restores the earlier state, preserving consistency.

Section F: Short Notes (any 5 marks each)

Q18. Write short notes on: (a) Integrity constraints (b) View (c) Functional dependency (d) Exception handling in PL/SQL [5 Marks]

- **Integrity constraints:** rules that keep data correct: NOT NULL, UNIQUE, PRIMARY KEY (unique + not null), FOREIGN KEY (referential integrity: value must exist in parent table), CHECK (condition), DEFAULT. Example: *CHECK (marks BETWEEN 0 AND 100)*.
- **View:** a virtual table defined by a SELECT query; stores no data itself. Benefits: security (hide columns), simplicity (hide joins), logical data independence. *CREATE VIEW v AS SELECT name, salary FROM Emp WHERE dept_id=10;*
- **Functional dependency:** X -> Y means each value of X determines exactly one value of Y. Types: trivial, non-trivial, partial, transitive, full. Basis of normalization; inferred by Armstrong's axioms (reflexivity, augmentation, transitivity).
- **Exception handling:** EXCEPTION block handles run-time errors. Predefined: NO_DATA_FOUND, TOO_MANY_ROWS, ZERO_DIVIDE; user-defined via RAISE / RAISE_APPLICATION_ERROR; WHEN OTHERS catches everything and SQLERRM gives the message.

Exam strategy (20 marks): Attempt questions you know best first. Spend about 1 minute per mark. Write definition first, then types/steps, then syntax/example, then advantage/difference. Underline keywords such as ECA, :OLD/:NEW, %NOTFOUND, partial dependency, transitive dependency, lossless join.

QUICK REVISION SHEET

Topic	Remember
SQL	DDL (CREATE, ALTER, DROP, TRUNCATE) | DML (INSERT, UPDATE, DELETE) | DQL (SELECT) | DCL (GRANT, REVOKE) | TCL (COMMIT, ROLLBACK, SAVEPOINT). WHERE before group, HAVING after group.
Trigger	Auto-fired on event. ECA model. BEFORE / AFTER / INSTEAD OF. Row-level (:OLD, :NEW) vs Statement-level.
Cursor	Pointer to query result area. Implicit (SQL) vs Explicit. Steps: DECLARE, OPEN, FETCH, CLOSE. Attributes: %FOUND, %NOTFOUND, %ISOPEN, %ROWCOUNT.
Normalization	1NF atomic | 2NF no partial dep | 3NF no transitive dep | BCNF every determinant is super key | 4NF no MVD | 5NF no join dep.
Procedure / Function	Procedure: action, optional return via OUT. Function: must RETURN one value, usable in SELECT. Modes: IN, OUT, IN OUT.

Answer-writing tips for 5-7 marks

- Start with a 1-2 line definition.
- Give classification / types in a table or points.
- Always include syntax and one short example.
- Add advantages (and disadvantages / differences) at the end.
- Draw a neat block diagram or table wherever possible.